

AEGIS — Autonomous Evidence-Gated Intake & Governance System

CIPHER 2.0 · Scenario 03 · Phase 2 Prototype · Team Amateurs, Informatics Institute of Technology

Evidence-weighted capstone team allocation with auditable governance **ALL DATA SYNTHETIC**

1 Executive Summary

AEGIS replaces manual, opinion-driven capstone allocation with an evidence-gated, fully auditable pipeline. The *same engine* that powers the demonstration runs live against a **70-student cohort**, forming **15 balanced teams + 1 exception pool**, flagging a near-duplicate proposal at **cosine 0.911**, and raising **6 governed monitoring alerts** — with every state-changing decision recorded in an append-only, hash-chained audit log. (All headline figures live — §5.2.)

The problem. Allocating students to capstone projects and teams is, in most institutions, a manual and opinion-driven exercise. Skills are taken from self-declaration at face value — so a confident over-claimer outranks a quiet expert; near-duplicate project topics slip past tired reviewers; and disengagement or free-riding only surfaces at the final viva, when it is too late to intervene. None of it leaves an auditable trail, so allocation decisions cannot be defended when challenged.

What AEGIS does. AEGIS is an evidence-gated pipeline — intake → de-duplicate → match → form → monitor — that treats every self-declared skill as a *claim to be weighted by its evidence*, not a fact. Unproven claims are discounted, near-duplicate projects are blocked for review before allocation, teams are formed to lift the weakest team rather than the average, and engagement is monitored continuously so faculty are alerted to ghosting, sympathy-carrying and burnout while there is still time to act.

What it is. A pure Python decision engine driving a Next.js dashboard over a Supabase / Postgres database, with object-scoped Row-Level Security and an append-only, hash-chained audit log as the governance substrate. Every state-changing action is recorded; every allocation is explainable.

2 The Algorithm

2.1 Evidence weighting — the Dunning–Kruger correction

Each declared skill is adjusted by a confidence factor reflecting the strength of its evidence:

$$\hat{A} = L \times C$$

where **L** is the declared level (1–5) and **C** is the confidence factor keyed to the evidence basis:

Evidence basis	C	Meaning
verified	1.0	institutional grade / assessed record
portfolio	0.8	portfolio or prior work submitted
self-report	0.6	self-report only (default; no LMS link)
contradicted	0.5	self-report contradicted by observed throughput

The **contradicted** tier is the Dunning–Kruger correction: when a high self-claim conflicts with measured task throughput, confidence is halved.

Demonstration scenario — STU_08. In the 12-student engineered cohort (§5.1), STU_08 declares Technical **5.0** with no supporting evidence, contradicted by throughput → $C = 0.5 \rightarrow \hat{A} = 5.0 \times 0.5 = 2.5$. The inflated claim is corrected before it can distort team formation; STU_08 ultimately lands in the critical team P_05.

2.2 Duplicate detection — TF-IDF cosine gate

Project abstracts are vectorised with TF-IDF and compared by cosine similarity. Any pair scoring ≥ 0.75 is flagged and blocked for supervisor review before allocation proceeds.

Demonstration scenario. In the engineered cohort (§5.1), projects **P_02** and **P_03** flag at **0.96** — well above the 0.75 gate — and are held for review rather than silently allocated as two distinct projects. On the 70-student live cohort (§5.2) the same gate flags a near-duplicate pair at **0.911**.

2.3 Matching — Student–Project Allocation (SPA)

Allocation uses the Abraham–Manlove Student–Project Allocation algorithm, honouring students' ranked project preferences against supervisor and project-capacity constraints — a stable matching rather than first-come allocation.

2.4 Formation — maximin floor-lifting

Teams are formed to maximise the capability of the *weakest* team (a maximin objective), rather than the average — deliberately lifting the floor so no team is left non-viable. The resulting health-band spread on the 12-student engineered cohort (§5.1):

Team	Health score	Band
P_04	84	Healthy
P_01	69	At-Risk — carry imbalance + burnout
P_05	41	Critical — ghost + overload + low completion

2.5 Monitoring — four engagement signals

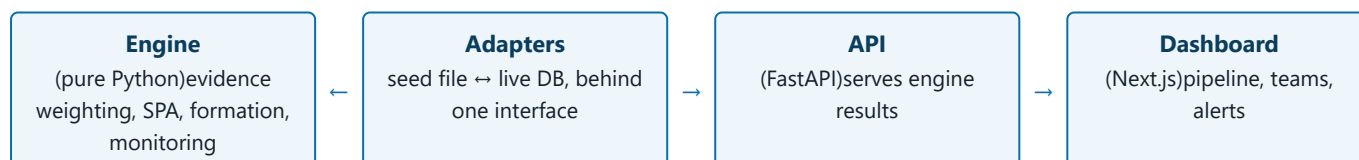
Over a 14-day sprint window the engine raises four governed signals:

- **Ghosting (Tier-3):** 10+ consecutive zero-input days → faculty critical alert.
- **Sympathy-carry:** one member authoring ≥ 0.95 of another's assigned tasks.
- **Burnout:** individual utilisation $\geq 2\times$ the team average.
- **Health-band alerts:** team score crossing into At-Risk or Critical bands.

Demonstration scenario. In the engineered cohort (§5.1), **STU_07** (team P_05) authors **zero events across all 14 sim-days** — a pure ghost, exceeding the 10-day Tier-3 threshold. In team P_01, **STU_01** authors ≥ 0.95 of STU_05's assigned tasks (sympathy-carry), and STU_01's resulting 10-event load trips the **burnout** tripwire — carrier and carried surfaced together.

3 Architecture

AEGIS is layered so that decision logic is isolated from data sources and transport:



- **Engine purity.** The engine imports *nothing* from the adapter layer — it operates on plain domain objects, so it is unit-testable on fixed data with no database in the loop.
- **Adapter env-switch.** A single environment switch flips the data source between the synthetic seed and the live Supabase database *without any change to engine code* — the same logic runs in demo and production.
- **Governance substrate.** Supabase Postgres with Row-Level Security enforces who can read which rows; the API never returns data the caller is not entitled to.
- **Role-based dashboard.** One Next.js app renders by role from the authoritative `profiles.role`: a **student** sees only their own team workspace; a **lecturer** sees cohort-wide monitoring (teams, health, alerts); an **admin** additionally sees the governance console (§4).
- **Drive integration.** Per-team Google Docs workspace provisioning (folder + report / tracker / deck) runs on OAuth-as-user with least-privilege scopes (§4). The activity-ingestion read path is **built**, but live engagement attribution is **deferred**: on a consumer-account share the Drive Activity API exposes an actor id, not an email, so faculty-grade attribution needs a Google Workspace Shared Drive (§6). Health today runs on representative seed activity.

4 Security & Governance

Controls are framed against the standards actually applied, with evidence rather than assertion.

- **OWASP API1:2023 (BOLA) remediation.** The `team_monitoring` surface was hardened from authentication-only to **object-scoped RLS**: an authenticated user can no longer read another team's monitoring rows — access is checked against row ownership, not merely a valid session.
- **OWASP API5:2023 (function-level authorization).** The governance API's `/admin/*` endpoints run on the `service_role` backend and therefore bypass RLS, so they are gated by a **single server-side dependency applied uniformly to every admin route** (no route can be added ungated). A caller must present a **token-verified identity whose authoritative `profiles.role` is `admin`** — the same column RLS trusts, never client-supplied metadata; the trusted backend writer authenticates with the `service_role` key, constant-time compared. Evidence (automated tests): an unauthenticated caller → 401, an authenticated non-admin → 403, an admin token → 200 — the API is not a way around RLS.
- **Three-tier role boundary — verified live in the running app.** Function-level authorization is demonstrated end-to-end, not only unit-tested: a **student** reaches only their own workspace; a **lecturer** reaches cohort monitoring (teams, health, alerts) but **not** governance; an **admin** reaches full monitoring **plus** the governance console (audit log, approvals, integrity badge). `/admin` and `/governance` redirect any non-admin away from the panel. This three-tier boundary was confirmed by an **independent browser audit** of the live application — authorization enforced in the running product, not merely at the API edge.
- **Three-tier RLS verification.** Verification distinguishes *toggle-exists* ≠ *policy-exists* ≠ **policy-enforces**. RLS being "on" is necessary but not sufficient; enforcement is confirmed at the row level, not at the toggle.
- **Privilege-escalation controls.** The `audit_log` is **append-only and hash-chained** (each entry binds the previous, so tampering is detectable). User **role is hard-forced server-side** — a client cannot assign itself a privileged role.
- **Least-privilege Drive.** The integration requests only `drive.file` + `drive.activity.readonly`, acting as the user via OAuth. The app can touch only the files it itself created — never the user's wider Drive.
- **CIS Control 3 (data protection).** No credential material exists anywhere in git history (verified by a targeted full-history pattern sweep for JWT/PEM/OAuth-secret/DB-URL signatures — see `PUBLICATION_CHECKLIST.md`). Seed data is synthetic only, using **RFC 2606 reserved `@aegis.test`** addresses that cannot resolve to a real mailbox.
- **Authoritative role of record.** UI authorization reads the same `profiles.role` the backend (`require_admin`) and RLS `is_admin()` trust — never client-supplied `user_metadata`. The dashboard resolves a signed-in user's role from `profiles.role` (read-only, own row), so the view a user lands on cannot diverge from what the server authorizes.
- **Data endpoints — scoping (OWASP API1/API5).** The read/compute routes (`/run`, `/teams`, `/alerts`, `/students/{id}`) are not individually authenticated; only `/admin/*` carries the authorization dependency. At prototype scale this is contained by compensating controls: the API binds to a private port (`:8000`), CORS is restricted to localhost / private-LAN origins, and the destructive service-role write-back is **opt-in and off by default** (`AEGIS_PERSIST` unset → an unauthenticated `/run` never writes to the live DB). Extending the same token-verified dependency to these routes is the documented production step (§6).

5 Results

The *same engine code* runs over two cohorts, selected by a single environment switch (§3): an engineered demonstration seed that exercises every mechanism, and the live Supabase cohort that shows production scale.

5.1 Demonstration scenario — 12-student engineered cohort

A hand-built 12-student / 8-project seed, **engineered so each formula fires visibly** — not a random sample. It forms 3 full teams with an empty exception pool, and surfaces exactly one case per mechanism:

Mechanism	Engineered evidence (seed)	Outcome
Evidence weighting	STU_08: declared 5.0, contradicted → $\hat{A} = 5.0 \times 0.5 = 2.5$	Inflated claim corrected (amber)
Duplicate detection	P_02 / P_03 cosine = 0.96 (≥ 0.75 gate)	Blocked for review
Ghosting (Tier-3)	STU_07: 0 events / 14 sim-days	Faculty critical alert
Sympathy-carry + burnout	STU_01 authors ≥ 0.95 of STU_05's tasks; 10-event load	Carry + burnout alerts
Formation spread	P_04 = 84 / P_01 = 69 / P_05 = 41	Healthy / At-Risk / Critical bands

STU_01–STU_12 and **P_01–P_08** are synthetic seed identifiers that exist *only* in this demonstration cohort — engineered to make each mechanism fire on cue. They do not appear in the live cohort.

5.2 Scale result — 70-student live cohort

The identical engine runs end-to-end against the **live Supabase database** over a **70-student cohort**, producing **15 teams + 1 exception pool**. Students are keyed by **UUID** rather than STU_ identifiers, so the demonstration's named probe cases (STU_07, STU_08, ...) do not exist here. The duplicate gate flags a near-duplicate project pair at **cosine 0.911** ($\geq$ the 0.75 gate) and holds it for review — the same governance behaviour as the seed, demonstrated at production scale.

Health band — live 70-student cohort	Teams
Healthy	11
At-Risk	4
Critical	0

This live distribution — **11 Healthy / 4 At-Risk / 0 Critical** (live) — is deliberately distinct from the engineered seed's one-of-each spread of **84 / 69 / 41** (§5.1), and the contrast *is* the point of the two-run framing. The seed is hand-built so every failure mode (ghost, sympathy-carry, burnout, a critical-band team) fires on cue for inspection; the live run shows the *same engine* producing **well-formed teams at scale with no critical-band team**. The two number sets are never interchangeable: the STU_/P_ identifiers and 84/69/41 belong only to the seed (§5.1); the 70 / 15+1 / 0.911 / 11-4-0 figures belong only to the live run (§5.2).

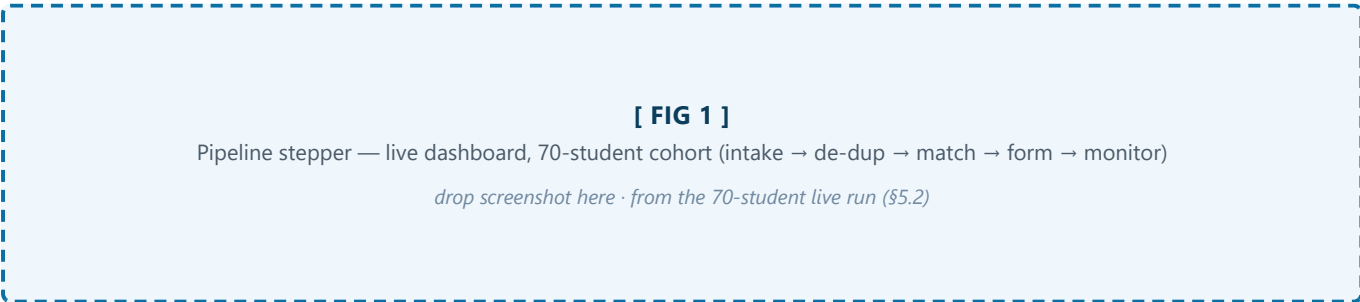
5.3 Two-role flow (live)

The same live data drives both roles at once. A **student** signs in and sees only their own team's workspace and health; a **lecturer or admin** signs in on a separate account and monitors all **15 teams**, the health bands, the alert inbox, and — for an admin — the governance console. Both sessions run simultaneously on different accounts without interference; the role boundary that separates them is the live-verified control in §4.

5.4 Judge access — demo accounts

Sign in at the dashboard login page with email + password (no Google sign-in required). These are **throwaway demo accounts on synthetic data** (@aegis.test, RFC 2606) — they correspond to no real person and are revoked after judging.

Role	Email	Password	Lands on
Student	judge.student@aegis.test	AegisJudge-Student-2026	Own team workspace
Lecturer	judge.lecturer@aegis.test	AegisJudge-Lecturer-2026	Cohort monitoring (no governance)
Admin	judge.admin@aegis.test	AegisJudge-Admin-2026	Full monitoring + governance



[FIG 3]

Alert inbox — STU_07 ghost (Tier-3) + STU_01/STU_05 carry & burnout

drop screenshot here · from the 12-student demonstration run (§5.1)

6 Limitations & Scaling

Documented, not built (deliberate prototype scope)

- **LMS grade sync** (the verified-evidence T1 channel) is mocked; with no institutional API the system defaults to $C = 0.6$ self-report and works without it.
- **Server-side caching / job queue** (e.g. Redis, BullMQ) for high-volume alert dispatch — noted as a scale-up path, not built at prototype scale.
- **Off-peak batch re-similarity sweeps** across the full cohort, versus the current per-submission duplicate check.

Honest engineering caveats

- **Non-atomic migrations.** Schema migrations are applied in manual order with no down-migrations; a failed mid-sequence apply needs manual reconciliation.
- **Schema-drift risk.** The synthetic seed shape and the live DB shape are maintained in parallel; they can drift if a migration lands on one path but not the other.
- **Drive ownership constraint.** On consumer Gmail a service account cannot own files (zero storage quota), so provisioning uses OAuth-as-user. A Google Workspace **Shared Drive** is the institution-grade path once that access exists.
- **Unauthenticated data routes (post-submission).** The engine read/compute routes are not yet per-request authenticated (only the governance API is); production extends the same token check to them, with the dashboard sending its session JWT. Mitigated at prototype scale by port isolation, restricted CORS, and write-back off by default.
- **Governance fail-open (fix-first, post-submission).** On a live error the governance endpoints currently fall back to seed data, so a broken live audit feed could render a green “Verified” badge. The fix is to fail **closed** — return an explicit degraded state (HTTP 503) rather than silently serving seed for an integrity read. (Backlog S2.)
- **Drive provisioning not idempotent (post-submission).** `POST /api/drive/provision` creates a new folder each call; re-posting for a team that already has one makes a duplicate. The fix is to check the team's existing `drive_folder_id` first and short-circuit. Admin-only and called once at allocation time today, so not reachable in normal use. (Backlog D1.)
- **Drive activity polling auth (post-submission).** The Drive-activity poll endpoint authenticates with a shared scheduler secret rather than a per-user identity; production should scope and rotate that secret (or run polling entirely behind the service backend). Not user-reachable in the demo.
- **Content-Security-Policy for embeds (post-submission).** The embedded Google Docs iframes should be constrained by an explicit CSP `frame-src https://docs.google.com` rather than a wildcard — deferred as production hardening.

7 Data Statement

All personal data in this system is synthetic and was generated for demonstration; it corresponds to no real individual.